

건강한하루 — 토큰 사용량 · 비용 자동 계산

"AI 서비스 운영 및 비용 관리.md" 강의를 day04에서 만든 '건강한하루' AI상담(RF-3) 위에 실제로 적용한 RF-4(운영 대시보드) 구현이다.

day04가 "테스트시나리오를 pytest로 자동화"했다면, day05는 "LLM 호출을 토큰/비용으로 자동 계측"한다.

```
day05_5일차/프로젝트/  
  backend/      # day04 backend + RF-4 사용량 계측/집계/알림 (app/core/pricing.py,  
                # app/models/usage_log.py, app/services/usage_service.py,  
                # app/routers/usage.py)  
  frontend/     # day04 frontend + 운영 대시보드 화면 (ui/usage_dashboard.py)  
  test/         # day04 테스트(TS-01~10) 그대로 + TS-11~13(사용량 계측/집계/알림)  
  report/      # export_usage_to_excel.py - 토큰/비용 리포트 엑셀 생성
```

1. 무엇이 자동으로 계산되나

AI상담(`POST /chat`)을 호출할 때마다 `agent_service.ask()` 안에서 `usage_service.track_usage("chat")` 가 다음을 자동으로 기록한다(사람이 직접 토큰을 세지 않는다).

계측 항목	방법
입력/출력 토큰	LangChain <code>get_usage_metadata_callback()</code> — Tool 호출 루프로 LLM이 여러 번 불려도 모델별로 자동 합산
비용(USD)	<code>app/core/pricing.py</code> 의 모델별 단가표 × 토큰 수
도구(Tool) 호출 수	응답 메시지 중 <code>ToolMessage</code> 개수

계측 항목	방법
응답 시간	요청 시작~끝
성공/실패	예외 발생 여부(실패해도 반드시 기록됨)

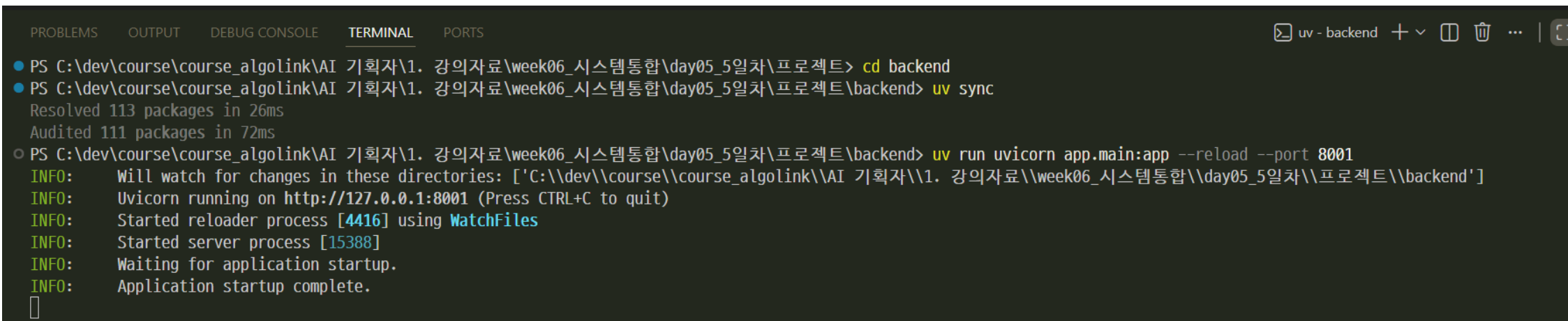
기록은 SQLite(`backend/data/건강한하루.db`)의 `llm_usage_log` 테이블에 쌓인다.
이 테이블은 `app/models/usage_log.py` 를 import하는 순간 없으면 자동으로 만들어지므로,
별도 마이그레이션이 필요 없다.

`backend/app/core/pricing.py` 의 단가는 예시값이다. 실습 전에 GROQ 콘솔의
최신 가격표로 반드시 갱신해야 한다("비용 산정의 기본식" 참고).

2. 실행 방법

backend

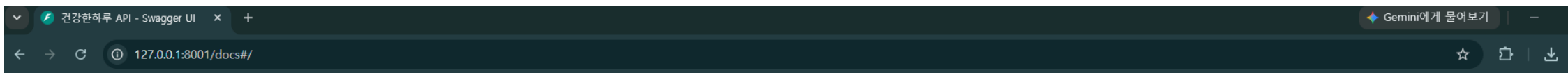
```
cd backend
uv sync
uv run uvicorn app.main:app --reload --port 8001
```



The screenshot shows a VS Code terminal window with the following content:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\dev\course\course_algolink\AI 기획자\1. 강의자료\week06_시스템통합\day05_5일차\프로젝트> cd backend
● PS C:\dev\course\course_algolink\AI 기획자\1. 강의자료\week06_시스템통합\day05_5일차\프로젝트\backend> uv sync
  Resolved 113 packages in 26ms
  Audited 111 packages in 72ms
○ PS C:\dev\course\course_algolink\AI 기획자\1. 강의자료\week06_시스템통합\day05_5일차\프로젝트\backend> uv run uvicorn app.main:app --reload --port 8001
  INFO: Will watch for changes in these directories: ['C:\\dev\\course\\course_algolink\\AI 기획자\\1. 강의자료\\week06_시스템통합\\day05_5일차\\프로젝트\\backend']
  INFO: Uvicorn running on http://127.0.0.1:8001 (Press CTRL+C to quit)
  INFO: Started reloader process [4416] using WatchFiles
  INFO: Started server process [15388]
  INFO: Waiting for application startup.
  INFO: Application startup complete.
```

접속



건강한하루 API 0.3.0 OAS 3.1

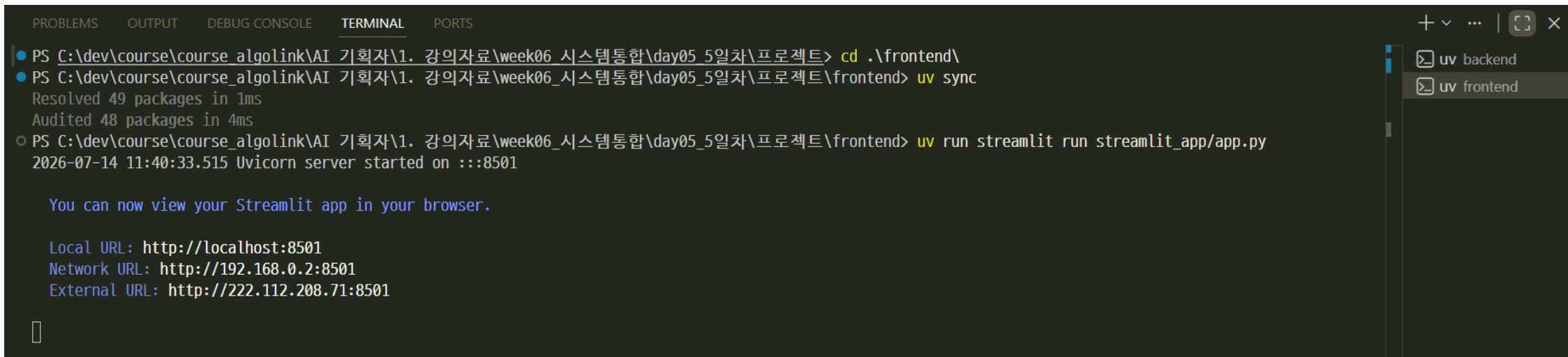
[/openapi.json](#)

건강기능식품 맞춤 추천 · 성분 설명 · 복용 정보 안내 서비스 (규칙 기반 + RF-3 AI상담 RAG Agent)

상품	▽
AI 맞춤 상품 추천	▽
AI 성분 설명	▽
복용 정보 안내	▽
RF-3 AI상담 (Agent)	▽
RF-4 운영 대시보드 (토큰/비용)	△
GET /usage/summary Read Usage Summary	▽
GET /usage/alerts Read Usage Alerts	▽

frontend

```
cd frontend
uv sync
uv run streamlit run streamlit_app/app.py
```



The screenshot shows a VS Code terminal window with the following content:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\dev\course\course_algolink\AI 기획자\1. 강의자료\week06 시스템통합\day05_5일차\프로젝트> cd .\frontend\
PS C:\dev\course\course_algolink\AI 기획자\1. 강의자료\week06 시스템통합\day05_5일차\프로젝트\frontend> uv sync
Resolved 49 packages in 1ms
Audited 48 packages in 4ms
PS C:\dev\course\course_algolink\AI 기획자\1. 강의자료\week06 시스템통합\day05_5일차\프로젝트\frontend> uv run streamlit run streamlit_app/app.py
2026-07-14 11:40:33.515 Uvicorn server started on :::8501

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.0.2:8501
External URL: http://222.112.208.71:8501
```

On the right side of the terminal, there is a sidebar with two tabs: "uv backend" and "uv frontend". The "uv frontend" tab is currently selected and active.

사이드바에서 "운영 대시보드" 메뉴로 들어가면 AI상담을 몇 번 사용한 뒤 일별 호출 수·토큰·비용·응답시간과 알림 상태를 바로 확인할 수 있다.

접속

건강한하루

AI 맞춤 영양제 추천

백엔드 연결됨

홈

건강고민입력

추천결과

상품상세

AI상담(고객센터)

운영 대시보드

마이페이지 — MVP 이후 제공 (Won't have)

Deploy

운영 대시보드 — 토큰 사용량 · 비용

AI상담(RF-3) 호출마다 자동으로 기록된 토큰 사용량과 예상 비용을 집계해서 보여줍니다. 실제 청구 비용이 아니라 backend/app/core/pricing.py에 등록된 단가로 계산한 예상치입니다.

조회 기간(일)

14

현재 알림 기준을 초과한 항목이 없습니다.

최근 14일 누적

누적 호출 수12

누적 비용(USD)\$0.0118

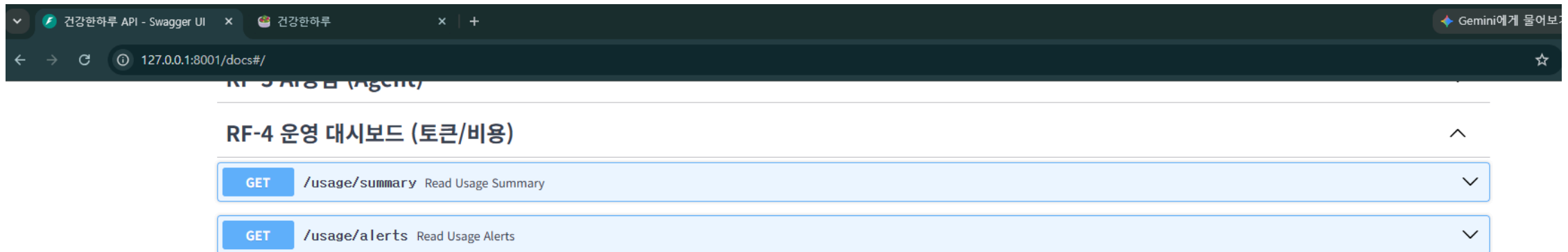
누적 실패율0.0%

일별 사용량

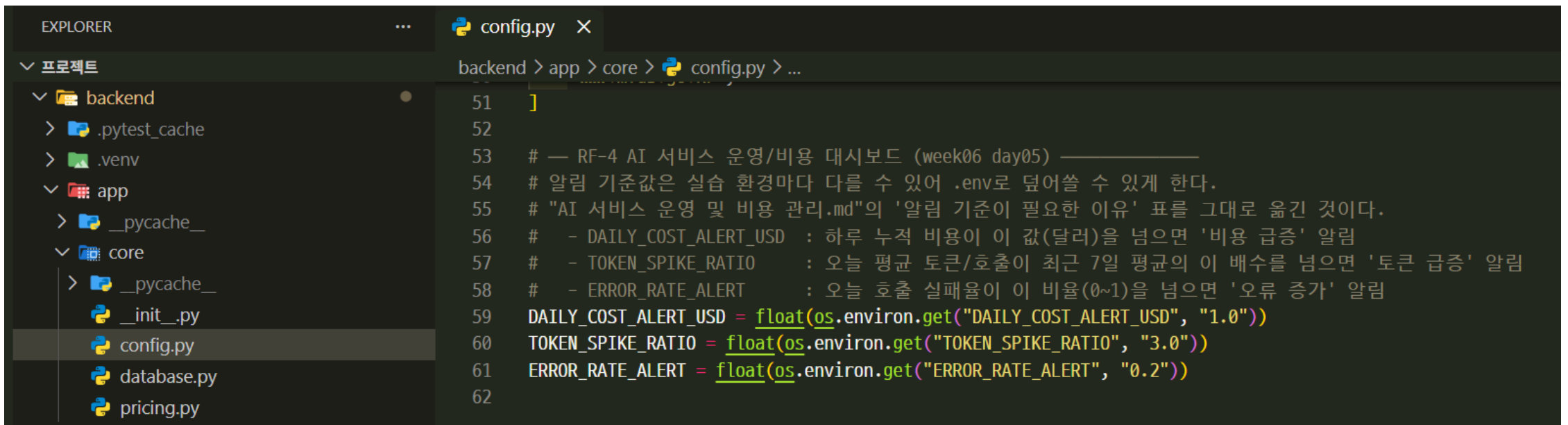
날짜	호출_수	성공_수	실패_수	입력_토큰	출력_토큰	총_토큰	총_비용_USD	평균_응답시간_ms
2026-07-14	12	12	0	47511	6197	53708	0.0118	35309.7

3. 집계 · 알림 API

API	설명
GET /usage/summary? days=14	최근 N일 일별 호출 수/토큰/비용/평균 응답시간/실패율
GET /usage/alerts	비용 급증 / 토큰 급증 / 오류 증가 여부 (기준값은 .env 로 조정)



알림 기준값(backend/app/core/config.py):



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with folders 'backend', 'app', and 'core'. The 'core' folder is expanded, showing files like '__init__.py', 'config.py', 'database.py', and 'pricing.py'. The 'config.py' file is selected. The code editor shows the following content:

```
51 ]
52
53 # — RF-4 AI 서비스 운영/비용 대시보드 (week06 day05) —————
54 # 알림 기준값은 실습 환경마다 다를 수 있어 .env로 덮어쓸 수 있게 한다.
55 # "AI 서비스 운영 및 비용 관리.md"의 '알림 기준이 필요한 이유' 표를 그대로 옮긴 것이다.
56 #   - DAILY_COST_ALERT_USD : 하루 누적 비용이 이 값(달러)을 넘으면 '비용 급증' 알림
57 #   - TOKEN_SPIKE_RATIO    : 오늘 평균 토큰/호출이 최근 7일 평균의 이 배수를 넘으면 '토큰 급증' 알림
58 #   - ERROR_RATE_ALERT    : 오늘 호출 실패율이 이 비율(0~1)을 넘으면 '오류 증가' 알림
59 DAILY_COST_ALERT_USD = float(os.environ.get("DAILY_COST_ALERT_USD", "1.0"))
60 TOKEN_SPIKE_RATIO    = float(os.environ.get("TOKEN_SPIKE_RATIO", "3.0"))
61 ERROR_RATE_ALERT     = float(os.environ.get("ERROR_RATE_ALERT", "0.2"))
62
```

환경변수	기본 값	의미
DAILY_COST_ALERT_USD	1.0	하루 누적 비용(USD)이 넘으면 '비용 급증'
TOKEN_SPIKE_RATIO	3.0	오늘 평균 토큰/호출이 최근 7일 평균의 이 배수를 넘으면 '토큰 급증'
ERROR_RATE_ALERT	0.2	오늘 호출 실패율이 이 비율을 넘으면 '오류 증가'

4. 엑셀 리포트 생성

```
# day05_5일 차 / 프로젝트 루트에서
uv run --project backend python report/export_usage_to_excel.py --days 30
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + - [ ] [ ] ... | [ ] [X]
● (healthy-day-api) PS C:\dev\course\course_algolink\AI 기획자\1. 강의자료\week06_시스템통합\day05_5일차\프로젝트> uv run --project backend python report/export_usage_to_excel
.py --days 30
[완료] '토큰_비용_리포트.xlsx' 생성 - 일별 요약 1행, 호출 상세 12행
○ (healthy-day-api) PS C:\dev\course\course_algolink\AI 기획자\1. 강의자료\week06_시스템통합\day05_5일차\프로젝트> [ ]
```

report/토큰_비용_리포트.xlsx 에 '**일별 요약**'(강의자료 '운영 지표 예시' 표와 동일한 구성)과 '**호출 상세**'(원자료) 두 시트를 새로 만들어 저장한다.

11. 토큰_비용_리포트

.XLSX

☆

📁

☁

파일

수정

보기

삽입

서식

데이터

도구

도움말

🔍

↶

↷

🖨

🔗

100%

W

%

0.0

0.00

123

기본값 ...

-

11

+

B

I

↶

A

🔗

📊

🔗

≡

⬇

↶

↷

↶

:

E23

fx

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	시각(UTC)	기능	모델	단가 확인	입력 토큰	출력 토큰	총 토큰	도구 호출 수	응답시간(ms)	비용(USD)	성공 여부	오류 메시지		
2	2026-07-14 00:35:08	chat	openai/gpt-oss-120b	Y	642	184	826	0	1909	0.0002343	성공			
3	2026-07-14 00:35:06	chat	openai/gpt-oss-120b	Y	646	132	778	0	8035	0.0001959	성공			
4	2026-07-14 00:34:58	chat	openai/gpt-oss-120b	Y	23143	2157	25300	8	190798	0.0050892	성공			
5	2026-07-14 00:31:46	chat	openai/gpt-oss-120b	Y	6805	467	7272	2	34298	0.001371	성공			
6	2026-07-14 00:31:12	chat	openai/gpt-oss-120b	Y	634	103	737	0	20668	0.00017235	성공			
7	2026-07-14 00:30:51	chat	openai/gpt-oss-120b	Y	1758	374	2132	1	16335	0.0005442	성공			
8	2026-07-14 00:30:35	chat	openai/gpt-oss-120b	Y	1692	573	2265	1	16330	0.00068355	성공			
9	2026-07-14 00:30:18	chat	openai/gpt-oss-120b	Y	2610	386	2996	1	37071	0.000681	성공			
10	2026-07-14 00:29:41	chat	openai/gpt-oss-120b	Y	2610	596	3206	1	37613	0.0008385	성공			

+

≡

일별 요약

호출 상세

5. 테스트 실행

day04의 TS-01~10 테스트는 그대로 있고(회귀 확인용), RF-4 관련 TS-11~13이 추가됐다. 실제 GROQ API를 호출하지 않고 LangChain의 `FakeMessagesListChatModel` (진짜 콜백 체인이 동작하는 가짜 ChatModel)로 결정적으로 검증하므로, 비용·시간이 들지 않고 `RUN_LIVE_AI_TESTS` 도 필요 없다.

```
cd backend
uv run pytest -v -k "ts11 or ts12 or ts13"
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell - backend + v [ ] [ ] ... | [ ]
```

```
(healthy-day-api) PS C:\dev\course\course_algolink\AI 기획자\1. 강의자료\week06_시스템통합\day05_5일차\프로젝트\backend> uv run pytest -v -k "ts11 or ts12 or ts13"
```

```
test_ts11_usage_logging.py::test_llm_call_records_usage_row PASSED [ 10%]
test_ts11_usage_logging.py::test_llm_call_failure_is_recorded_as_failed PASSED [ 20%]
test_ts11_usage_logging.py::test_multiple_calls_in_one_request_are_aggregated_per_model PASSED [ 30%]
test_ts12_usage_summary.py::test_get_daily_usage_groups_and_sums_by_date PASSED [ 40%]
test_ts12_usage_summary.py::test_get_summary_aggregates_daily_rows PASSED [ 50%]
test_ts12_usage_summary.py::test_usage_summary_endpoint_returns_expected_shape PASSED [ 60%]
test_ts13_usage_alerts.py::test_cost_alert_triggers_when_threshold_exceeded PASSED [ 70%]
test_ts13_usage_alerts.py::test_no_alerts_when_thresholds_not_exceeded PASSED [ 80%]
test_ts13_usage_alerts.py::test_error_rate_alert_triggers_on_high_failure_rate PASSED [ 90%]
test_ts13_usage_alerts.py::test_usage_alerts_endpoint_returns_expected_shape PASSED [100%]
```

```
===== 테스트 시나리오(TS)별 결과 요약 =====
=====
TS-11 [PASS] Pass 3 / Fail 0 / Skip 0 (총 3건) - LLM 호출마다 토큰 사용량/비용이 자동으로 기록된다
TS-12 [PASS] Pass 3 / Fail 0 / Skip 0 (총 3건) - 일별 토큰/비용 사용량이 정확히 집계된다
TS-13 [PASS] Pass 4 / Fail 0 / Skip 0 (총 4건) - 비용·토큰·오류율 알림 기준을 넘으면 알림이 발생한다
===== 10 passed, 23 deselected in 1.23s =====
```

```
(healthy-day-api) PS C:\dev\course\course_algolink\AI 기획자\1. 강의자료\week06_시스템통합\day05_5일차\프로젝트\backend>
```


TS ID	시나리오
TS-11	LLM 호출마다 토큰 사용량/비용이 자동으로 기록된다
TS-12	일별 토큰/비용 사용량이 정확히 집계된다
TS-13	비용·토큰·오류율 알림 기준을 넘으면 알림이 발생한다

전체 회귀는 day04와 동일하게 `uv run pytest -v` (AI상담 라이브 테스트는 `RUN_LIVE_AI_TESTS=1` 일 때만 실행됨 — [test/backend/README.md](#) 참고).